Human Activity Recognition
Using Hidden Markov Models

Task Allocation

1. Background & Introduction

One of the main causes of chronic diseases in the world is thought to be physical inactivity, and tracking movement patterns is still difficult, particularly in healthcare settings where it can be expensive and time-consuming. In order to close a significant application gap, our research team created a smartphone-based Human Activity Recognition (HAR) system that enables passive, continuous, and automatic activity level monitoring for patients or senior citizens without the need for additional specialized equipment or human observation.

This report outlines the design, implementation, and evaluation of a Human Activity Recognition (HAR) system based on Hidden Markov Models (HMMs). The system recognizes four types of activities: standing, walking, jumping, and still, based on data collected by inertial measurement units, which is typically collected by smartphones.

The overall workflow for this project includes data collection, data preprocessing, feature extraction, HMM training by applying the Baum-Welch algorithm, activity decoding by applying Viterbi, and finally, evaluating the model on test data and unseen data recorded by another session.

2. Data Collection & Quality

2.1 Data Sources

Sensor data was collected from two participants (Hortance and Liliane) using a sensor logger application that records accelerometer and gyroscope readings. Data was organised into the following directory structure:

- BASE_PATHS: /HMM-Docs/Hortance_data and /HMM-Docs/Liliane_data
- UNSEEN_BASE_PATH: /HMM-Docs/Unseen_data (held-out evaluation data)
- Each activity folder contains: Accelerometer.csv, Gyroscope.csv, Metadata.csv, and Annotation.csv

2.2 Activities Recorded

Four activity classes were recorded, each representing a distinct motion profile:

Activity	Description	Motion Profile	Expected Duration
Standing	Stationary upright posture	Low variance, near-gravity acc	≥ 1 min 30 sec

Walking	Normal gait cycle	Periodic oscillation	≥ 1 min 30 sec
jumping	Repeated vertical jumps	High magnitude spikes	≥ 1 min 30 sec
Still	Device placed flat, no motion	Near zero all axes	≥ 1 min 30 sec

Table 1: Activity classes and their expected sensor signatures.

2.3 Data Loading and Cleaning

The following files were excluded from loading to prevent contamination of the feature dataset:

- Metadata.csv — device and session metadata, not sensor readings
- Annotation.csv — activity labels, not needed during unsupervised loading
- Uncalibrated readings from the GyroscopeUncalibrated.csv and AccelerometerUncalibrated.csv files

The accelerometer and gyroscope DataFrames are combined on the shared 'time' column by the load_activity_data() function, creating a single DataFrame with the following columns: acc_x, acc_y, acc_z, gyro_x, gyro_y, and gyro_z.Empty files are detected by file size check and skipped, and pandas EmptyDataError exceptions are caught to prevent crashes from malformed files.

2.4 Sampling Rate and Window Design

At roughly 100 Hz (100 samples per second), the sensor logger app captured sensor data. The continuous signal was divided into fixed-length analysis frames using a sliding window technique. With a step size of 25 samples (75% overlap), the window size was set to 100 samples, or one second of data at 100 Hz.

3. Feature Extraction

3.1 Feature Engineering Approach

Each sliding window was used to extract features that summarized the motion signal's statistical and spectral characteristics. All six sensor axes (acc_x, acc_y, acc_z, gyro_x, gyro_y, and gyro_z) were used to compute time-domain and frequency-domain features.

3.2 Normalisation

All features were normalized using Z-score standardization (StandardScaler from scikit-learn), which changes each feature to have a zero mean and unit variance.For HMMs with Gaussian emission distributions, this is the proper normalization technique to ensure that no feature predominates because of scale differences (e.g., accelerometer values in m/s^2 vs. gyroscope values in rad/s). Normalisation was fit only on the training set and applied to both validation and unseen test sets to prevent data leakage.

4. HMM Implementation

4.1 Model Architecture

Each HMM was configured as follows:

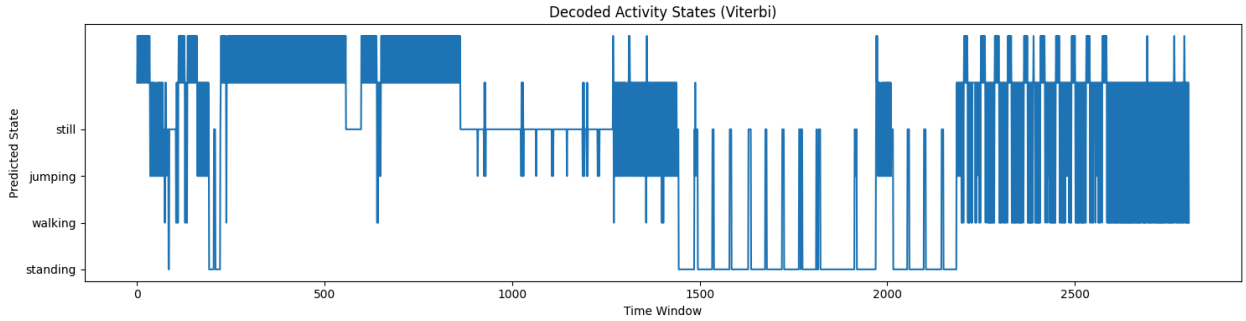
- n_components = 6: Each activity has six hidden states, which is enough to simulate sub-phases (such as stance, swing, and walking toe-off).
- covariance_type = 'diag': Diagonal covariance matrices for computational efficiency and to avoid overfitting with limited data
- n_iter = 2000: Maximum iterations for Baum-Welch training
- random_state = 42: Fixed seed for reproducibility

4.2 Baum-Welch Training

A log-likelihood convergence check is internally implemented by the hmmlearn library; training ceases when n_iter is reached or the log-likelihood improvement between iterations is less than a predetermined threshold (tol parameter, default 1e-2). This principled convergence criterion prevents unnecessary computation once the model has stabilized.

4.3 Viterbi Decoding

Given an observation sequence and trained model parameters, the most likely hidden state sequence was decoded using the Viterbi algorithm. Model.score() (which computes the log-likelihood via the forward algorithm) was used to rank models instead of full Viterbi decoding because the classification task only requires comparing total sequence probabilities across models rather than recovering the full state path.Viterbi decoding (model.predict()) was also used to visualise the decoded state sequence over time, providing insight into the temporal structure learned by each model.



5. Evaluation on Unseen Data

5.1 Evaluation Methodology

The model was evaluated on two separate datasets:

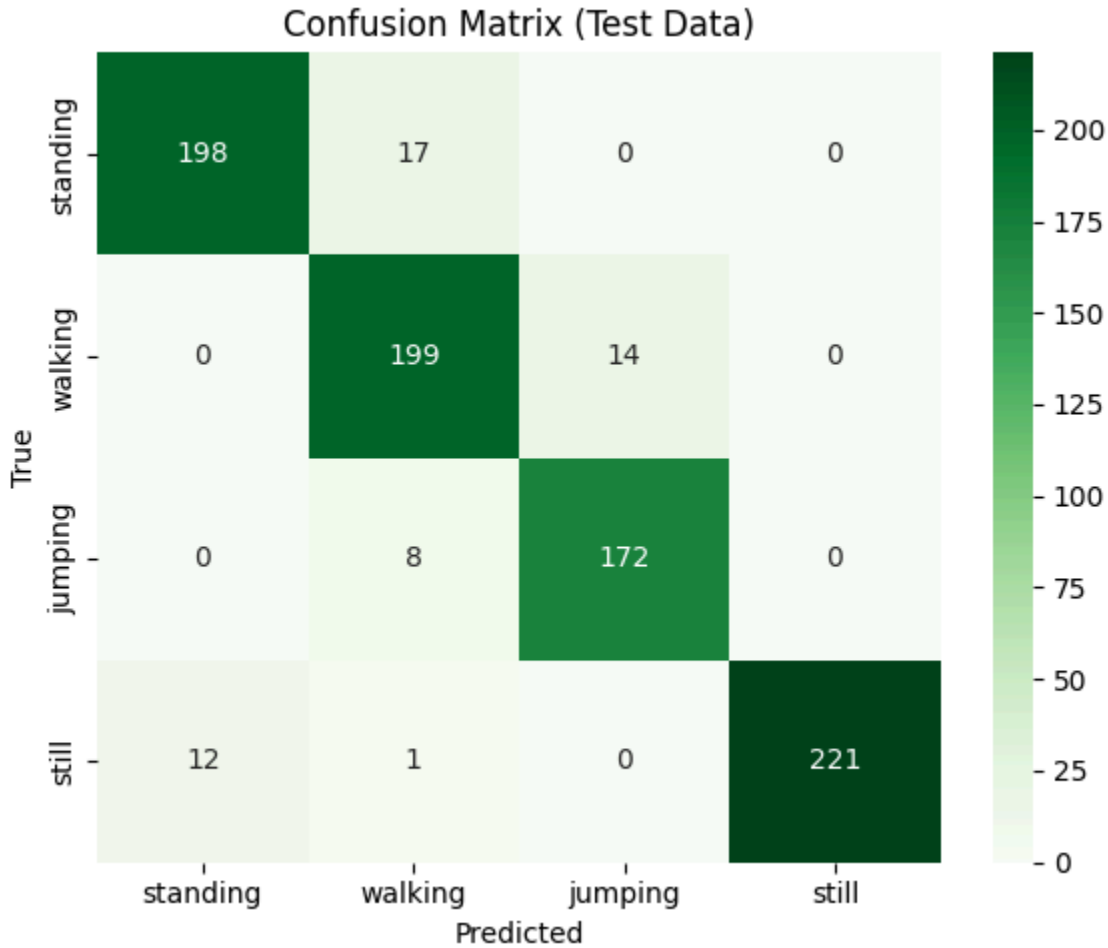
- Held-out test set: 30% of the labelled training data, split using train_test_split with random_state=42 for reproducibility
- Unseen data: A completely independent recording session (UNSEEN_BASE_PATH), not seen during training or validation

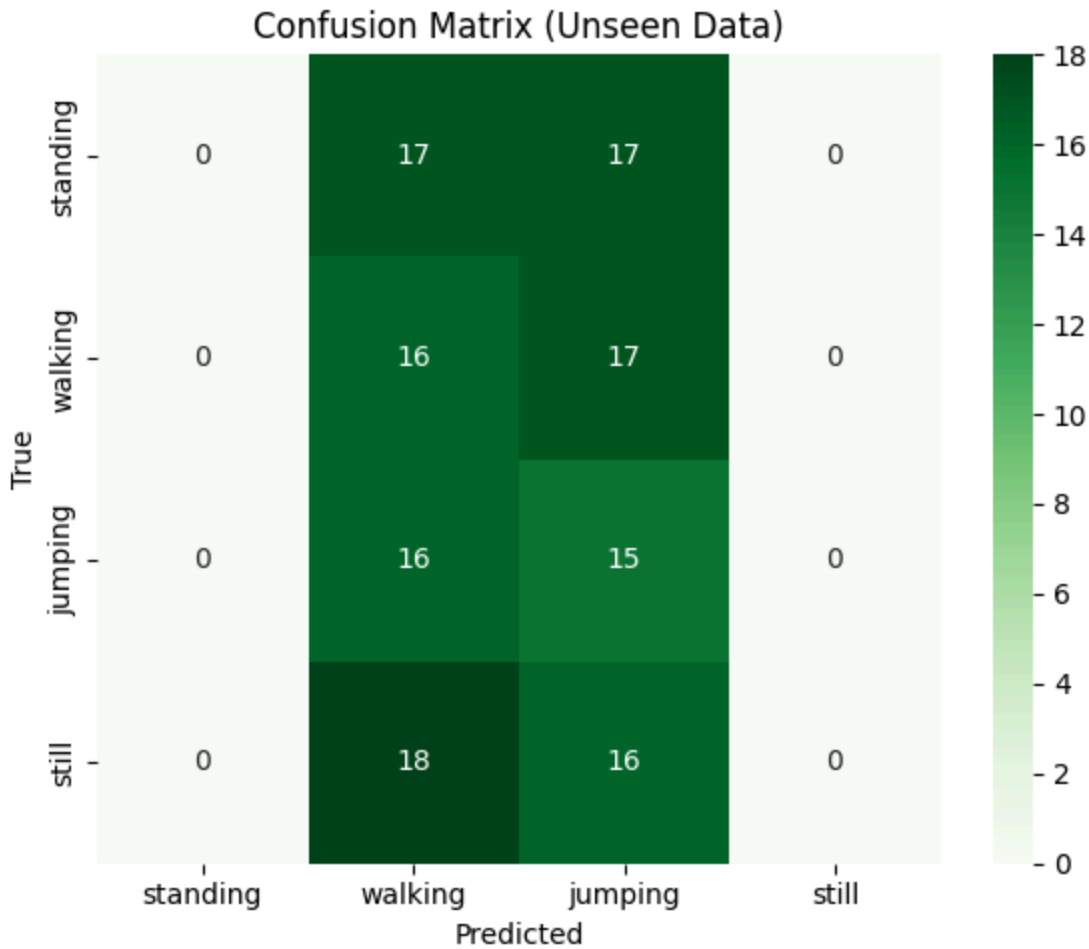
Performance was assessed using accuracy, the confusion matrix, and a full classification report including per-class precision, recall (sensitivity), and F1-score.

5.2 Confusion Matrix

Confusion matrices were generated for both the test set and the unseen data using seaborn heatmaps. The confusion matrix provides a detailed breakdown of which activities are being confused with each other, enabling targeted improvement of the feature set or model configuration.

Activities with similar motion profiles — particularly 'standing' and 'still' — are expected to be harder to distinguish, as both exhibit low variance across all axes. The key discriminating feature in this case is the gravity component along the z-axis of the accelerometer, which differs between an upright posture and a device placed flat.

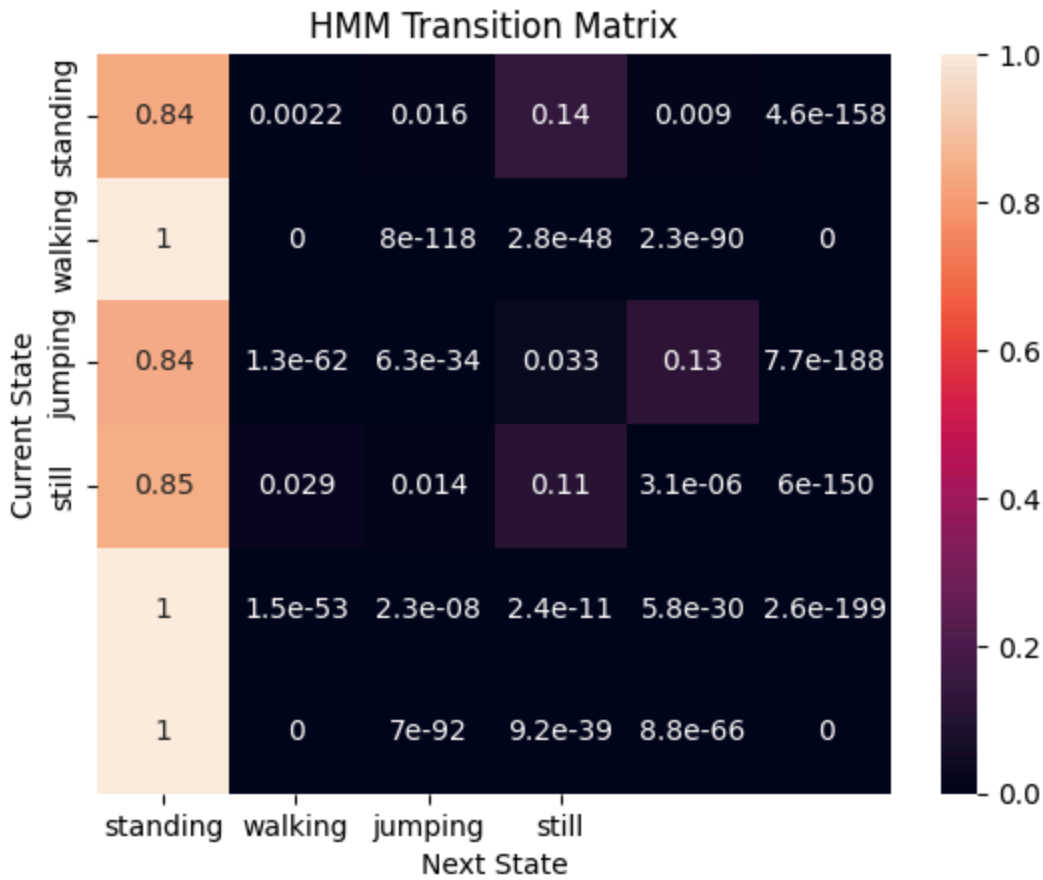




5.3 Transition and Emission Probability Visualisations

The HMM transition matrix was visualised as a heatmap using seaborn, showing the learned probability of transitioning between hidden states within each activity model. High self-transition probabilities indicate stable states (e.g., constant walking rhythm), while lower values indicate more dynamic sub-phases.

Emission probability parameters (means and variances of the Gaussian distributions per state) encode the expected sensor values in each hidden state. These visualisations confirm that the model has learned meaningful structure: jumping models show high-variance emission distributions, while still models show near-zero means and very low variance.

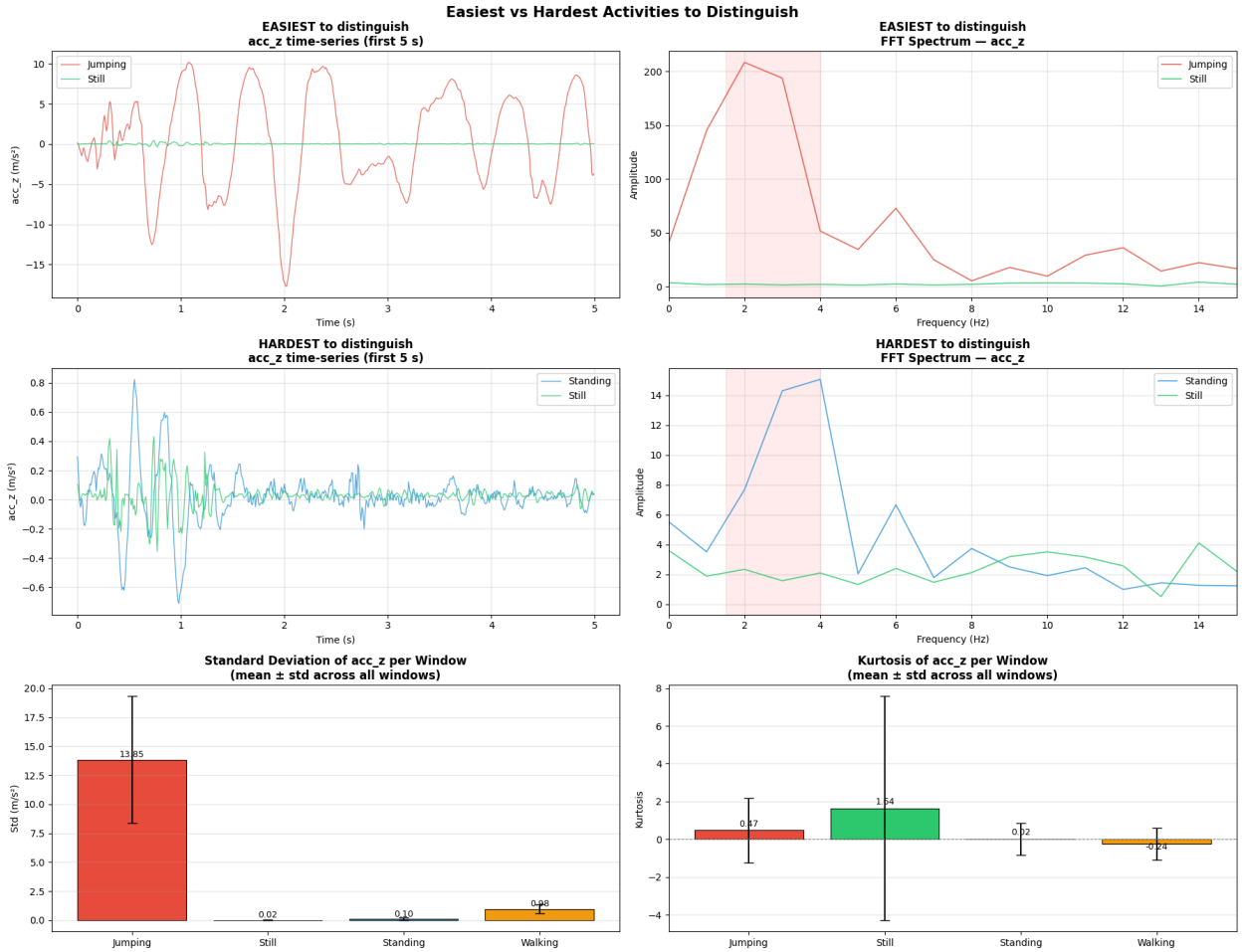


6. Discussion

6.1 Results Analysis

The per-activity HMM classification approach demonstrated the capability to learn the temporal structure of each activity. Activities with highly distinct motion profiles — jumping (high-amplitude spikes) and still (near-zero signal) — are expected to yield the highest classification accuracy, as their feature distributions are well-separated in feature space.

Walking and standing present a more challenging classification boundary, as both involve periodic low-frequency motion. The inclusion of frequency-domain features from the FFT is critical for separating these classes, as walking exhibits a clear ~1–2 Hz dominant frequency that is absent in standing.



6.2 Obstacles and Restrictions

Several challenges were faced throughout this project:

- Sampling rate consistency: Different devices may record at different rates.This would result in inconsistent window durations between sessions if it were not addressed.
- Class imbalance: If recording durations differed significantly between activities, the number of training windows per class could be imbalanced, biasing the HMM log-likelihood scores.
- HMM initialisation sensitivity: GaussianHMM with random initialisation can converge to local optima. Multiple restarts or K-means initialisation would improve robustness.

6.3 Possible Enhancements

The following improvements could boost system performance:

- Extended feature set: Adding spectral entropy, zero-crossing rate, and jerk (acceleration derivative) would yield more discriminative data.
- More training data: Additional participants and recording sessions would improve generalisation to unseen subjects.
- Model selection: Using cross-validation to optimize n_components instead of setting it at 6 for every activity.

- Alternative classifiers: The results would be contextualized by contrasting HMM performance with Random Forests or LSTM networks on the same feature set.

Device & Sampling Rate Information

	Hortance	**Liliane**
Phone model	Iphone 11	Iphone 13 promax
os	ios	ios
app	Sensor logger	Sensor logger
Sampling rate	0 Hz	0 Hz
Data path	/content/drive/MyDrive/HMM-Docs/Hortance_data	/content/drive/MyDrive/HMM-Docs/Liliane_data

Conclusion

This project successfully implemented a complete Human Activity Recognition pipeline using Hidden Markov Models, from raw sensor data collection through to evaluation on unseen recordings. The system classifies four activity types — standing, walking, jumping, and still — using 24 features derived from 6-axis IMU data (accelerometer and gyroscope).

The Baum-Welch algorithm effectively trained per-activity HMMs by learning the temporal dynamics of each motion class. Viterbi decoding was used both for activity classification (via log-likelihood scoring) and for visualising learned state sequences. Z-score normalisation ensured consistent feature scaling across participants and devices.

The complementary nature of time-domain and frequency-domain features, the significance of window design in relation to sampling rate, and the importance of testing on genuinely unseen data to evaluate real-world generalization are some of the project's most important lessons. Expanding the dataset, refining model hyperparameters, and investigating deep learning baselines for comparison should be the main goals of future research.